# TECNOTS

## AI ENGINEER INTERNSHIP

## Technical Assessment

*Supplementary Question Paper*

| | |
|---|---|
| **Candidate Name** | |
| **Date** | |
| **Question** | Question 1 — AI Document Q&A Knowledge Assistant |
| **Total Marks** | 50 marks (+ up to 10 bonus) |
| **Suggested Duration** | 2 Hours |
| **Allowed Resources** | AI tools, internet, IDE of choice |

*This question is mandatory and self-contained — worth 50 marks.*

## Instructions to Candidates

Please read the following carefully before you begin.

- This is a build question. You are expected to deliver a functional, working product — not a written design.
- You may use any AI tools, coding assistants, or online resources. This is intentional. We want to see how you work with AI, not without it.
- Submit the working code along with a brief explanation of your decisions.
- Marks are awarded for your reasoning, approach, and code quality — not just the final output.
- A deployed link or screen recording of your working product is not required but will be viewed favorably.
- Creativity and product thinking matter as much as technical correctness.

# QUESTION 1 — BUILD: AI DOCUMENT Q&A KNOWLEDGE ASSISTANT

**[50 MARKS]**

## The Task

Build an AI-powered knowledge assistant that lets a user upload a collection of documents and ask questions about them in plain English. The system must answer only from the uploaded content, show where each part of the answer came from, reason across multiple documents when a single one is insufficient, and surface disagreements between sources rather than silently choosing one. It must refuse to invent information that is not present.

**This is not a theoretical exercise. You are expected to deliver a functional, working product that ingests real documents and answers real questions about them.**

## What the Product Must Do

### Part 1 — Document Ingestion

The user must be able to upload one or more documents from their device. The application must accept:

- PDF files
- Plain text files (.txt) and Markdown (.md)

On upload, the system must process each document (extract text, split it into retrievable chunks, and index it) and give the user clear confirmation that it is ready to query. The user must be able to see a list of all loaded documents and remove any of them. If an unsupported or unreadable file is selected, a clear and specific error message must be shown.

Ingestion must work for any document set the user provides — a folder of research papers, several versions of a policy, or competing supplier contracts — without changes to the codebase.

### Part 2 — Natural Language Question Input

The user must be able to type a question in plain English. The interface must support direct lookups, synthesis, and comparison across documents.

Examples of inputs the product must handle:

- What is the refund window?
- Summarise the tenant's obligations under this agreement
- Which of these two contracts has the longer notice period?
- In the 2024 handbook only, what is the leave policy?

### Part 3 — Retrieval and Grounded Answer Generation

For each question the system must retrieve the most relevant passages, generate an answer using only the retrieved content, and present it clearly.

The AI must not use outside or general knowledge to fill gaps. If the documents do not contain the information, it must say so explicitly rather than guess.

### Part 4 — Cross-Document Reasoning and Conflict Detection

When a question cannot be answered from a single passage, the system must combine evidence from multiple chunks or multiple documents to form one coherent answer.

When two or more sources give conflicting answers to the same question (for example, two policy versions stating different refund windows), the system must not silently pick one. It must present each conflicting value alongside the document it came from, and clearly flag that the sources disagree.

The user must also be able to scope a question to a specific document and have the system honour that scope.

### Part 5 — Source Attribution

Every answer must be accompanied by its supporting evidence. For each answer the system must show:

- Which document(s) the answer was drawn from
- The specific passage(s) or chunk(s) used to produce the answer
- The location within the source — page number for PDFs, or section/heading where available

The user must be able to verify the answer against the original source at a glance.

### Part 6 — Conversation View with Follow-ups

The interface must maintain the running conversation in the current session. The system must correctly resolve follow-up questions that refer back to earlier turns.

For example, after asking *“What is the refund window in Contract A?”*, a user asking *“And in Contract B?”* or *“Why is it different?”* must be understood in context, without the user repeating the subject. The user must be able to scroll back through previous question–answer pairs and revisit the sources cited for any earlier answer.

## Edge Cases You Are Expected to Handle

| Situation | Expected Behaviour |
|---|---|
| A question is asked but no documents have been uploaded | A clear message instructs the user to upload a document first |
| The answer is not contained in any uploaded document | The system states it cannot find the answer in the provided documents rather than fabricating one |
| Two documents give conflicting answers | Both values are shown with their sources, and the conflict is explicitly flagged |
| A follow-up relies on the previous turn for its subject | The reference is resolved from conversation context, not treated as a brand-new question |
| The uploaded file is corrupted, empty, or unsupported | A clear error message is shown and the user is prompted to try again |
| A document contains instructions aimed at the AI (e.g. “ignore previous instructions and…”) | The text is treated as data to be retrieved over, never as a command to follow |
| Retrieved passages are only weakly relevant | The system answers cautiously or indicates low confidence rather than overstating certainty |
| The question is empty or too short to be meaningful | Submission is blocked with a helpful prompt to ask a complete question |

## Bonus Features

*Attempt these only after the core product is complete and working.*

| Feature | Description |
|---|---|
| Streaming Answers | The answer renders token by token as it is generated rather than appearing all at once |
| In-Source Highlighting | The exact sentence(s) used are highlighted within the displayed source passage |
| Confidence Indicator | Each answer displays a High / Medium / Low confidence level based on retrieval relevance |
| Reranking | A second-stage reranker reorders retrieved passages before answer generation; explain the gain |
| Conversation Export | The user can download the full Q&A session, including cited sources, as a file |
| Retrieval Tuning Note | In a short written section (max 200 words), explain your chunking, retrieval, and conflict-handling strategy, why you chose it, and the trade-offs involved |

## Submission

- A link to a version-controlled repository with your complete code
- A readme file with: steps to run the project locally, technologies used and why you chose them (including your choice of embedding/vector store and language model), how you approached chunking, retrieval, and cross-document conflict handling, and any assumptions or trade-offs made
- Screenshots or a short screen recording of the working product

## Marks Breakdown — Question 3

| Area | Marks |
|---|---|
| Document ingestion — upload, multi-format parsing, chunking and indexing, validation | 9 |
| Grounded answer generation from a single source, with abstention when no answer exists | 12 |
| Cross-document reasoning — synthesis, conflict detection, document-scoped queries | 12 |
| Source attribution — correct documents, passages, and in-source location | 7 |
| Conversational follow-ups with correct coreference resolution | 5 |
| Edge case handling — no documents, no answer, conflicts, follow-ups, injection, empty input | 5 |
| **Total** | **50** |

*Bonus marks for the written retrieval section and optional features are awarded on top of the core 50, up to a maximum of 10 additional marks.*